

Coursework: Database + web interface project

Due date – 12th December 2025, submitted via Moodle

Coursework is INDIVIDUAL WORK

Coursework Deliverables

WARNING: If you submit any non-ZIP formatted archives instead of ZIPs (e.g., RAR) your submission may appear corrupt to markers and therefore parts of it could receive ZERO marks.

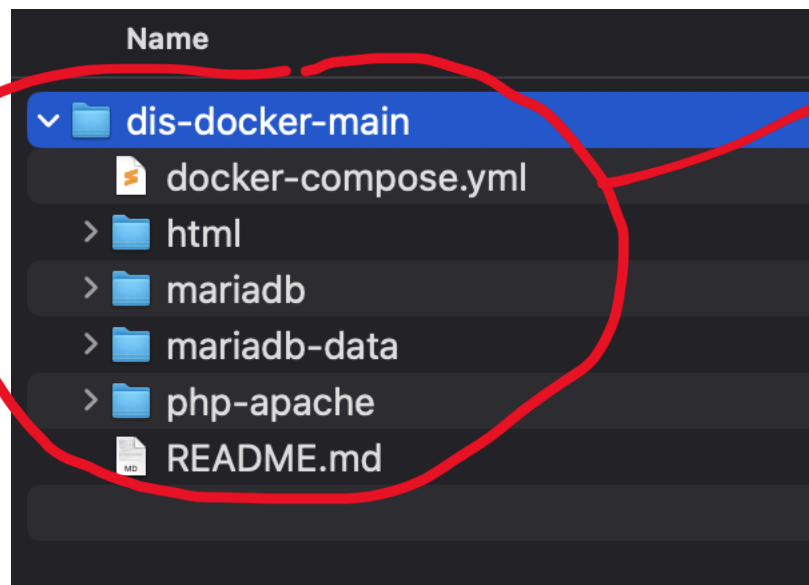
Overview. You must provide the following deliverables **in a single directory, ZIPped up** and submitted by the due date. You **must submit the following core directory structure** with the following labels (replacing `yourusername` and `studentID` with your university username and your student ID number respectively):

```
COMP4039-CW-yourusername-studentID/      [dir]
├── yourusername-studentID_CW_cover.txt    [txt file]
├── yourusername-studentID_Docker/        [dir]
│   ├── html/cw/index.php                  [php file]
│   ├── html/cw/...                        [other files]
│   ├── mariadb/cw-database.sql             [phpMyAdmin export]
│   └── mariadb-data/                      [dir]
├── yourusername_Technical.pdf             [pdf file]
└── yourusername_UserManual.pdf            [pdf file]
```

To summarise, you must include:

- A cover text called `yourusername`-`studentID`_CW_cover.txt
- `html/cw/index.php` as the entry point into the system
- The contents of your `mariadb-data` directory (this contains your working database)
- `mariadb/cw-database.sql` i.e., an export from phpMyAdmin of your Coursework database schema (that you have adapted from what was provided to you)
- Two manuals in PDF form

Hint:



Rename "dis-docker-main" to `yourusername-studentID_Docker` and put all of this in your submission!

Further details elaborating the Overview. In your ZIP file:

The cover text file (named `yourusername-studentID_CW_cover.txt`) must contain your name, your student ID and your username (please use **.txt / plain text file format**). In this file you should provide a list of all the core files you submitted for this coursework, i.e., including the full contents of the `html` directory. NOTE: There is no need to include a listing of the contents of `mariadb-data`!

`yourusername-studentID_CW_cover.txt` must also list & specify the origin of ALL resources (e.g., frameworks, assets, styles, JS libraries, code adapted / used from internet sources, code generators, etc.) that you have used that are not solely your own work. Please also include references to resources you have used in comments within your code. If you do not do so we may consider the absence of such a declaration as indicating academic misconduct.

The copy of your Docker directory (i.e., `yourusername-studentID_Docker`) must contain all necessary files so that it can be run on an equivalent Docker instance (i.e., based off the github Docker configuration provided). The safest way to ensure this is simply to rename `dis-docker-main` to `yourusername-studentID_Docker` and include the whole Docker directory in your submission, as noted above.

You must include the SQL definitions file that describes your database and data i.e., `mariadb/cw-database.sql`. You must also include your working database as found in the `mariadb-data` directory. This is so that we can reconstruct your database using phpMyAdmin or simply deploy it in our own Docker instance; including both forms ensures redundancy during marking. If you are unsure how to export your database, you will need to click on your database in phpMyAdmin to export in the left panel, then select the Export tab (if you are unsure still, use Google to search for this operation – it is very simple). **If SQL files are not present we may be unable to mark your coursework and you will receive ZERO marks.**

The two documentation files (**PDF only please or else these components will be marked ZERO**) are described in the specification below in greater detail.

Late submissions will be accepted, but they will be penalised at the University standard rate of 5% per working day.

It is assumed you have read and understood the module's Academic Misconduct Policy document.

Coursework Specification

The coursework scenario is described in the associated file in Moodle – **coursework-scenario.docx**. You will need a database definition and data, a draft version of which is provided in Moodle as **coursework-database.sql** (also available in your Docker in `mariadb/coursework-database.sql`). Note this will need modification and extension.

The task is to create a usable end to end system that will allow QMC doctors and administration to access the patient data. Although the database provided is small you should consider how your design solutions would scale to a much larger amount of data and incorporate additional requirements. It is important to provide a task-focussed interface that supports required *user tasks*, not simply an administration interface to database tables.

You must implement an interactive web frontend for your database using PHP, PHP's SQL interface, and HTML (as well as other technologies such as CSS or JavaScript, as you see fit).

This package / your system should be installable on **any** machine with a MySQL / Apache / PHP stack (e.g., LAMP)—so you **should avoid absolute pathnames** and you should use header files to ensure configuration (e.g., MySQL hostname etc.) is held in **one file only**. For example, see `db.inc.php` in the PHP demo code available on Moodle (use `require("path/to/db.inc.php")` in your other PHP files to include this common header).

We will be using **Google Chrome** (available on CS machines) to test your coursework (so make sure web interface elements function correctly in that browser).

Marking criteria

Coursework is worth a total of 70% of the total marks for this module.

Coursework is marked out of 100. The mark will be scaled to provide the final mark for this coursework. Most marks are allocated for specific required features, although to achieve the higher end of marking we encourage you to go beyond what is required. There are a few general mark categories that are awarded for the coursework as a whole which will also benefit from creatively **going beyond the bare minimum of functionality specified in the questions**. Finally, we note that marking will take into account specific code fragments (e.g., PHP, SQL, JS, etc.) that you have been provided with by convenors.

Marking criteria for features/requirements

0 (zero)	1-3	4-5	6-7	8-9	10
No attempt is made to implement this feature. No documentation	An attempt has been made to implement the feature, but it is significantly flawed in some way, and there is no evidence that this approach will fulfil the requirements of the specification. Flawed or very incomplete documentation	The approach is fundamentally sound, and produces the expected results, but may include some flaws, such as not producing the correct results. Functionality that mostly works but include some bugs or problematic methods of implementation would fall into this category. UI elements may have inconsistencies or other flaws. Basic documentation, although perhaps lacking structure or richness.	Implementation works and produces the expected results, meets and moderately exceeds core functionality required. UI elements are well designed and implemented correctly to support <i>flexible</i> interaction with the database, as well as other aspects such as error handling, feedback, etc. Underlying SQL takes full advantage of querying flexibility and combines multiple data sources where relevant. Documentation is well presented and clear.	Previous requirements are met, but offers substantive additional functionality beyond this. More advanced use of styling and dynamic elements (e.g., application of JS). Overall implementation and UI should be of fairly high quality, but it is likely to lack the polish of a professional solution. Documentation approaching professional standard.	Previous requirements are met, and the code and its structure are of very high quality, well documented and easily reusable. Implementation including UI of professional production quality, follow standard usability guidelines and practices. Goes significantly and creatively beyond core functionality. Documentation should be of production level quality.

Core required features [76 marks total]

NOTE: For each core required feature, relevant aspects of the UI *will* be taken into account, with an additional overall assessment provided under “Other required features” (below).

1. A doctor should be able to log into the system using one of the following usernames/password combinations:

Username	Password
mceards	lord456
moorland	buzz48

Once logged in the doctor should have the ability to manage their account with a reasonable number of basic functionalities, e.g., change their password, log out, look parking permit request status etc.

IMPORTANT NOTE – in any real-world system involving a database and web frontend, security would be paramount. However, addressing security is beyond the scope of this module, so you should ignore this. For example, usernames and passwords can be stored as plain text fields in your database – do not attempt to encrypt them or secure your database in any way.

[10 marks]

2. The doctor should be able to flexibly search for / look up patients by their names or their NHS number and retrieve information related to the patient(s). If the patient is not in the system, you should provide an appropriate message. The system will then show ward admission and test details and related information (e.g., the admission date, admission duration, tests performed, test performed dates etc.). Allow for missing data in the system (e.g., the patient might be in the system but not admitted and/or no test is performed).

[10 marks]

3. The doctor should be able to add a new test and then prescribe the added test to any patient. If the patient is already in the database, then you need to provide a way to associate that person to the new test as part of this process. If the patient is not in the database, they will need to be added as part of this process.

[15 marks]

4. The doctor should be able to update their profile and can request for a parking permit. Once the doctor will enter their car details, the parking permit fee should be displayed to them depending on the selected parking permit option (monthly/yearly). Once the doctors submit the parking permit request, the request will go to the administrator and administrator can then approve or reject the request.

[10 marks]

5. Administrators should be able to log into the system with a specified username (please include at least one user named "jelina" with the password "iron99"). Administrators should be able to do everything that an ordinary doctor can do with the following additions:
- Administrators should be able to create new doctor's account
 - Administrators should be able to approve/reject the parking permit request. If administrator approve the request, he needs to add parking permit number against each request. If administrator rejects the request, administrator should provide the reason for rejection.

[15 marks]

6. To support various regulatory and statutory requirements, the administrator need to have access to an audit trail to account for database record accesses and changes that are made (e.g., deletions, updates, etc.). Implement an auditing feature in the web interface which will enable administrators to review the above, on a per user basis (you may extend this to provide other auditing views e.g., per record, filtering by user, editing facilities, etc.).

[16 marks]

Other required features [24 marks total]

Documentation. You should produce two documentation manuals – a user manual and technical documentation. The *user manual* should explain to a user (i.e., a doctor or an administrator) how to use the system. It is extremely important that all of the software features you have implemented should be described in this manual (otherwise you might not get marks for them!). The user manual should be no more than 500 words, though it may have screen shots if you so wish.

The *technical manual* should contain all of the information that would be needed if you were handing this project over to another developer to take it further. It should briefly explain how to install your software (you may assume that MySQL, and a web server implementing PHP are already installed and configured). It should also contain a detailed description (including diagrams) of architecture / structure of your system, together with a brief rationale for its design. For example, you could include explanations of key flows of data / interaction, key PHP components, etc. Your technical manual must include an Entity-Relationship diagram to that represents your final database design. You should show all entities together with their attributes and the relationships between these entities. You should annotate the cardinality ratios of each relationship. You may use an ERD tool or similar, alternative digital option (e.g., Powerpoint), or draw by hand and photo (note that if your photo is illegible in some way you may lose marks). Note: Please avoid including screenshots of code or large amounts of code listings. The technical manual should be no more than 700 words (diagrams, screenshots, etc. do not count for words).

[14 marks]

User interface (overall assessment). This component of marking represents an assessment of the UI as a whole, including visual design, interaction design and usability. You might pick up a few marks for creating a beautiful user interface, but not many if it is not functional. What is more important is that your system is usable, in this case with minimal or ideally no training. We strongly advise you primarily examine [Nielsen's 10 heuristics](#) and secondarily look for existing best practices in web interface design to guide you.

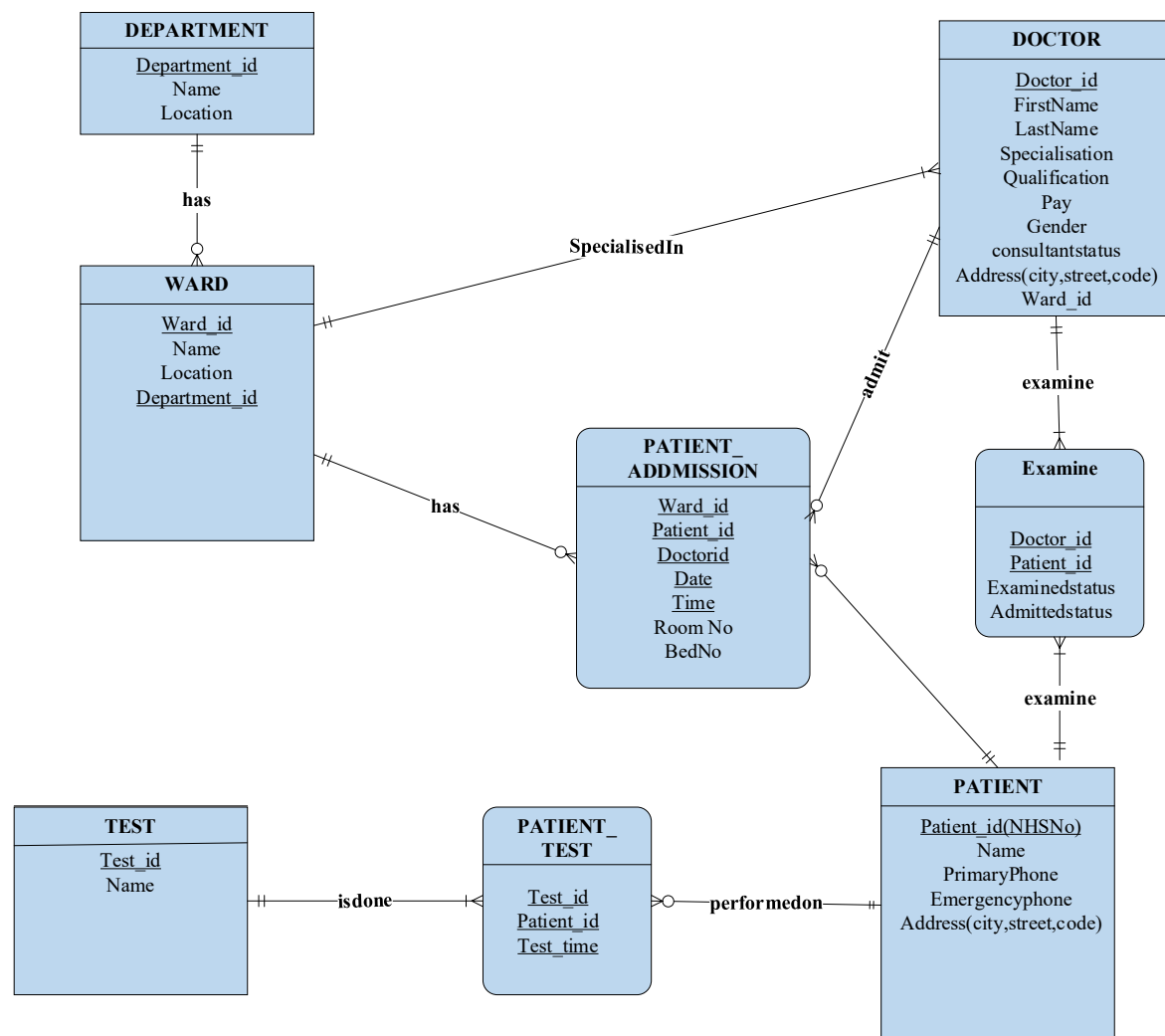
[10 marks]

Case Study:

The Queen's Medical Centre (QMC) stands as a prominent healthcare facility in Nottingham, England. Traditionally, the hospital managed its records through manual filing systems. However, the surge in patient numbers has rendered the maintenance of these paper records cumbersome. As a database designer, the initial task is to design a part of current system that can efficiently store and manage all the hospital records, addressing the challenges posed by the increasing volume of patient information. There are several specialized departments (such as Dermatology, Urology, Orthopaedics etc) in the Queens Medical Hospital. Every department has several wards. Information about ward includes unique name, phone, location, total numbers of current patients. Each ward hosts several patients, who were admitted by a consultant (doctors) employed by the Hospital. On admission, the date and time are kept. Also the patient status (admitted, discharged) is kept. The personal detail of every patient includes name, NHS number, primary phone, gender, emergency phone and address (city, street, code). Hospital also stores the information of the tests undertaken. Each test has unique test No., category (blood test, urine test, MRI etc),

and the result of test. Number of tests may be conducted for each patient and is prescribed by the doctor. Doctors are specialists in a specific ward and may be leading consultants for several patients. Each patient is assigned to one leading consultant but may be examined by other doctors, if required. The details of doctor stored in the database are staff no(NHS number) ,first name, last name, specialisation, qualification, pay, gender and address. Due to increased number of doctors request for car parking permit, QMC management also wants to automate their car parking system for doctors only. The doctors working at the hospital can request for a parking space. Only one parking permit could be allowed to each doctor and they can pay the parking fee either monthly or yearly and should pay in advance. For parking permits, parking id, doctor id, car registration number, permit choice (monthly or yearly), permit activation date, permit end date and amount needs to be stored.

Apart of the ERD (below) and database (SQL files) is already provided. As a database designer, your task is to update the ERD and database according to the additional requirements.



The End